toward content management. For the journalists, the content in question is newspaper articles. For a blog site, the content is a blog post. In both of these cases, the content is text that makes up the page, but that isn't the only choice.

An e-commerce website has pages of items for sale, a shopping cart, and payment processing. This isn't really considered a content management system, but it does have some things in common. The catalog pages query the database for available items and display them—the "content" here is the merchandise. A shopping cart is a special case of this, where the contents of the cart are specific to the user and this visit.

Consider what blog and e-commerce sites have in common: there are "things" stored in the database (blog posts or merchandise), and users visit pages that display one or more of those things. The Django ORM provides an object-oriented abstraction for database storage, making it easier to code and manage these things, and the Django Admin provides a web interface for interacting with instances of them. Django views use the ORM to show the user either the blog posts or merchandise appropriate to the page and construct the page through the template engine. Its general approach of objects plus views rendering templates makes Django a good fit for any multipage website.

1.2.1 Server-side, single-page, and mixed-page applications

As web applications have evolved, the architectural choices in building them has grown. Inside a Django view, the template engine gets called to render HTML to be sent to the browser. This is called *server-side rendering*. By contrast, a *single-page application* (SPA) moves this rendering and some application logic into the browser itself. An SPA's approach to the MVC model is for the view and controller parts to be entirely in the web browser.

Server-Side Rendering vs. SPAs

The phrase *server-side rendering* can be a little confusing. Of course, in all situations, your browser displays the HTML. However, the distinction between server-side rendering and SPAs is where the HTML is composed. In server-side rendering, the server is responsible for assembling all of the pieces of the HTML and sending it down to the browser. The template engine runs on the server side, composing the parts of the template into a result, like in figure 1.8.

By contrast, SPAs create the HTML in the browser. All the logic for page creation happens in the browser, so you can think of this as a browser-based template engine. The server sends the code for building the templates to the browser as well as the data necessary, but all page composition happens on the client side.

This doesn't mean Django can't be used to build SPAs. When a user visits an SPA-based site, a JavaScript bundle gets downloaded. The bundle contains the complete user interface and a lot of the business logic for the application. The JavaScript presents a dynamic interface hosted in a single web page, hence the name. The application still